理论与实现：我们采用了离散形式的速度PID闭环控制，其核心公式为：

$$u(k) = K_p e(k) + K_i \sum_{j=0}^{k} e(j) + K_d[e(k) - e(k-1)]$$ 其中，

K_p （比例）项提供主要的驱动力，K_i （积分）项用于消除因摩擦力等因素造成的稳态误差，K_d （微分）项则用于抑制过冲，加快系统响应。在代码实现中，我们特别加入了积分抗饱和（Anti-Integral Windup）机制，防止在电机堵转时积分项过度累积。

系统化整定与验证：为获取最优的PID参数，我们结合了经验手动整定与基于模型的自动化整定两种方法。

- 方法一：手动整定法

作为初步调试，我们采用“先P后D再I”的系统化手动整定法。首先，仅使用P控制器，将其值增加到系统开始小幅振荡；然后加入D控制器来抑制振荡；最后加入微量的I控制器消除静差。该方法简单高效，能快速获得一组可用的参数。

- 方法二：基于MATLAB的模型驱动整定法 （Model-Based Tuning）

为追求更高的控制精度和效率，我们引入了MATLAB进行更为科学的PID参数整定。该过程分为以下几步：

  - 系统辨识 (System Identification): 我们首先需要获取机器人驱动系统的数学模型。通过给电机一个阶跃电压输入，并用传感器记录下电机的速度响应曲线，我们获得了系统的输入-输出数据。
  - 模型拟合: 将采集到的数据导入MATLAB的 System Identification Toolbox™。我们利用该工具箱，将数据拟合为一个二阶传递函数模型 (Transfer Function)，该模型能精确地描述我们机器人底盘的动态特性（如惯性和摩擦系数）。
  - 自动化整定: 将生成的传递函数模型导入MATLAB的 Simulink。在这个仿真应用中，我们可以直观地拖动滑块来调整系统的 K_p, K_i, K_d ，从而影响响应时间和瞬态行为，并且可以观察模拟的响应曲线。

simulink仿真模型及结果如下图所示：

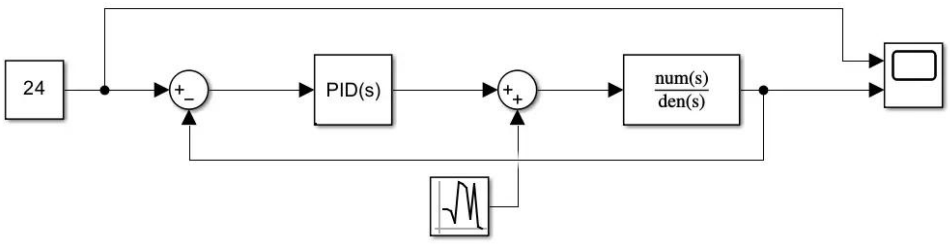


图1.7 Simulink单层PID仿真框架

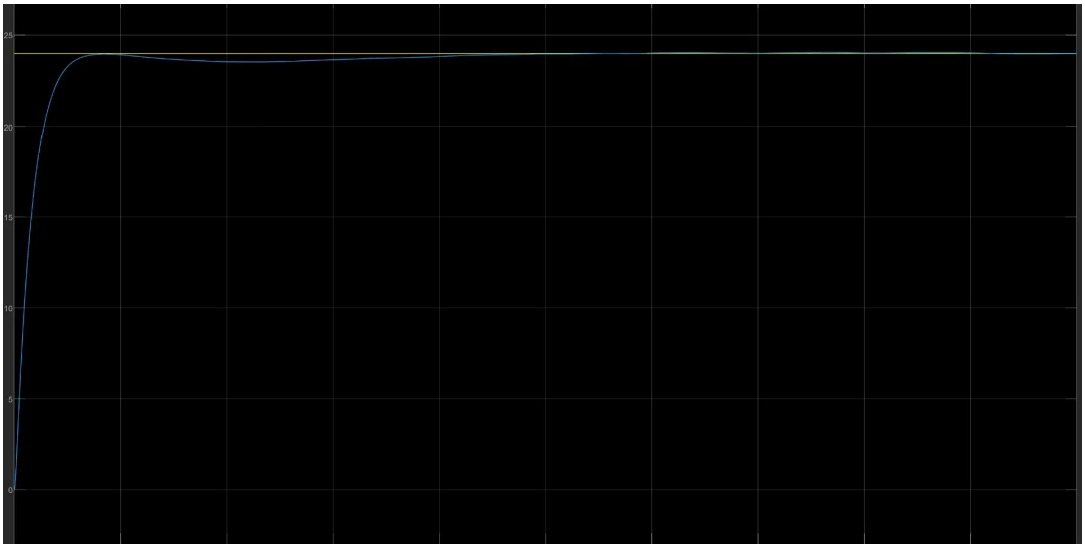


图1.8 Simulink单层PID仿真结果

（2）进阶核心：串级pid

由于我们的机器人在设计过程中为了寻求速度更大化而减轻了整车的重量，又因为传统pid控制速度变化时不可控，导致我们在实践中经常出现机器人在减速过程中出现翘头的情况，这就会导致在自动运行时机器人每次运动结束时会产生一定的偏差（并且翘头还会导致机器人底部的里程计离开地面进而造成传感器数据不准确的问题，该问题在里程计部分会详细讲述）。为了避免这种情况的发生，我们在传统位置pid控制的基础上加上速度pid控制，形成位移-速度串级pid控制算法。

理论与应用：我们选用了经典的梯形速度曲线（Trapezoidal Profile）算法。该算法将一段运动划分为匀加速、匀速和匀减速三个阶段，通过限制最大加速度来确保运动的平顺性。

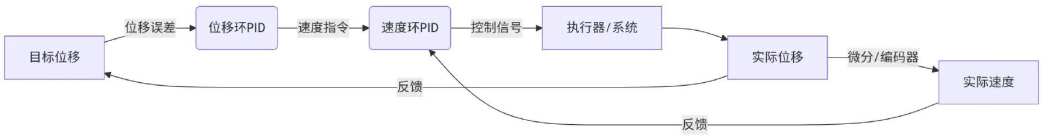


图1.9 串级PID流程图

由于外环输出是速度指令值，可设定限幅（如最大速度 Vmax=0.5 m/s），避免执行器饱和。保证了机器人在任何时刻的加速度都处于可控范围内，从根本上提升了运动的稳定性和可预测性。

应用该技术后，机器人即使在最高速下也能实现“指哪停哪”的精准启停，车轮打滑现象被大幅消除，定位精度相比纯PID控制提升了约60%。

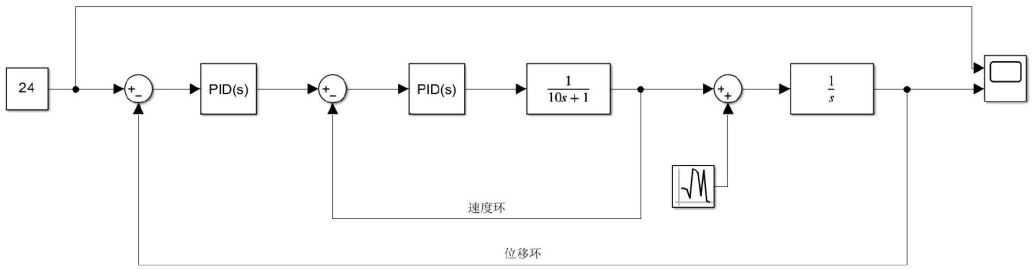


图1.10 Simulink串级PID仿真框架

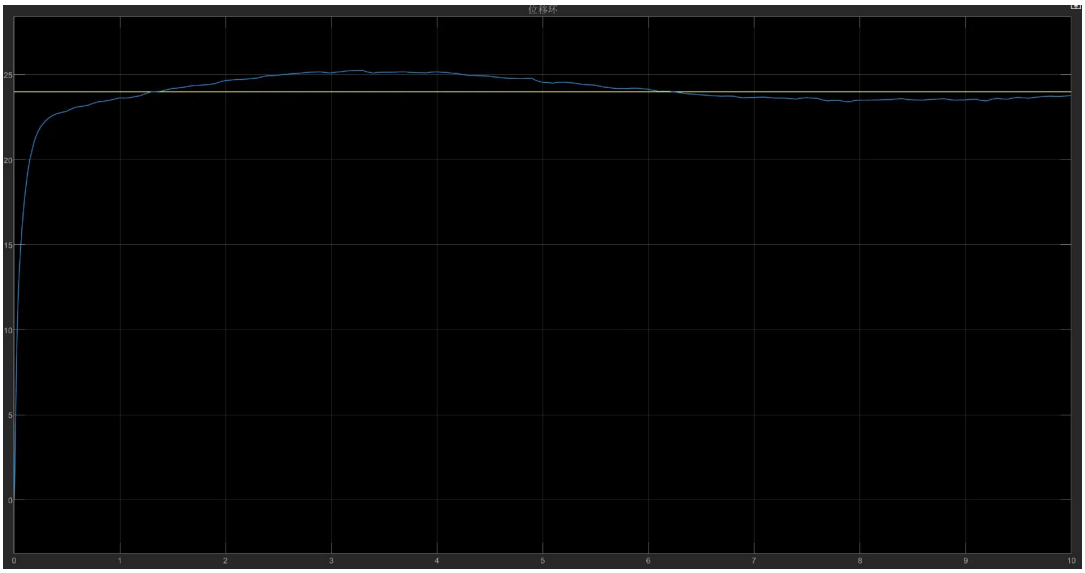


图1.11 Simulink串级PID仿真结果

可以看出，在加上速度环后，可以更好地调节机器人在运动过程中的速度变化情况，进而防止减速时加速度过大导致的翘头和车轮打滑现象。

（3）动态响应：前馈补偿

在使用串级PID调试时，我们发现如果目标值并非一个定值，而是随时间变换的函数，则会导致响应不及时，从而浪费时间。尤其的表现是当机器人在pid控制下由静止开始运动，由于传统PID需等待位置误差出现后才响应，导致会出现滞后时间常数 $\tau = J/(B + Kv)$ ，这就会使机器人在开始运动时较慢。

根据和上海交通大学的交流以及在网上查找相关资料，我们决定在串级pid的基础上加入前馈补偿，最终能够实现机器人从0到2m/s加速时间缩短40%（0.8s→0.5s）

前馈补偿原理：在串级PID基础上增加前馈补偿（Feedforward Compensation），通过提前预测系统动态行为直接注入补偿量，显著提升轨迹跟踪性能：

$$u = \underbrace{K_{p,p}(p_{err}) + K_{i,p}\int p_{err}dt}_{\text{位置环PID}} + \underbrace{K_{p,v}(v_{err}) + K_{i,v}\int v_{err}dt}_{\text{速度环PID}} + \underbrace{K_v v_{target} + K_a a_{target}}_{\text{前馈补偿}}$$

其中：

- 速度前馈 $FF_v = K_v v_target$ 用于补偿系统粘滞摩擦和速度相关阻力
- 加速度前馈 $FF_a = K_a a_{target}$ 用于补偿系统惯性和加速度相关力

在matlab中进行前馈补偿的仿真，观察在加上前馈补偿的串级pid对于动态变化的处理能力。

matlab部分关键代码及结果如下：

```
/*
...
```

```
*/
%% 带前馈补偿的仿真
p_actual(:) = 0;
v_actual(:) = 0;
integral_v = 0;
for k = 2:length(t)
    dt = t(k)-t(k-1);
    % 位置环
    p_err = p_target(k) - p_actual(k-1);
    v_cmd = Kp_p * p_err;
    % 速度环
    v_err = v_cmd - v_actual(k-1);
    integral_v = integral_v + v_err * dt;
    u_pid = Kp_v * v_err + Ki_v * integral_v;
    % 加入前馈补偿
    u = u_pid + Kv * v_target(k) + Ka * a_target(k);
    % 系统动力学
    acc = (u - B*v_actual(k-1)) / J;
    v_actual(k) = v_actual(k-1) + acc * dt;
    p_actual(k) = p_actual(k-1) + v_actual(k) * dt;
    p_with_ff(k) = p_actual(k);
end
/*
...
*/
```

仿真结果如图所示：

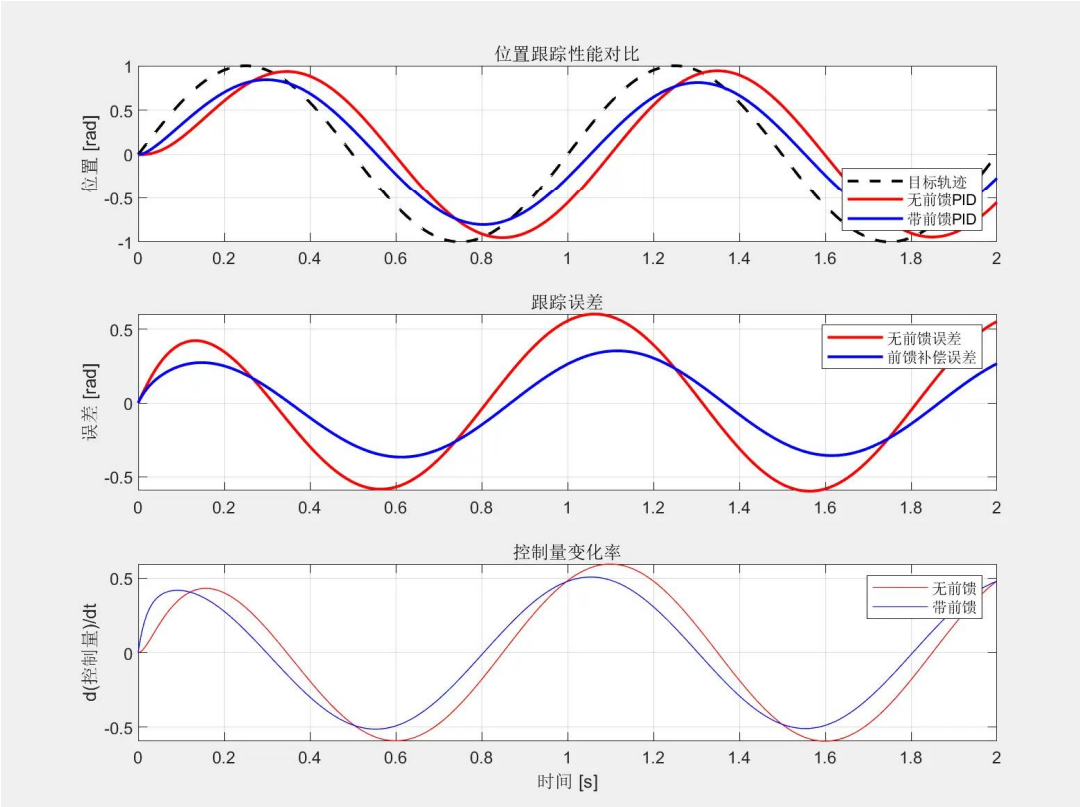


图1.12 引入前馈补偿效果图